

# Contents

|                                                                |   |
|----------------------------------------------------------------|---|
| Abstract .....                                                 | 4 |
| 1 Introduction.....                                            | 4 |
| 2 Fault Injection Methodology.....                             | 4 |
| 2.1 Bit-Flip Fault Injection.....                              | 4 |
| 2.1.1 Injection Logic .....                                    | 5 |
| 2.1.2 Automation and Execution.....                            | 5 |
| 2.1.3 Environment and Tooling .....                            | 5 |
| 2.2 Stuck-at Fault Injection.....                              | 5 |
| 2.2.1 Stuck-at-1 Logic.....                                    | 5 |
| 2.2.2 Stuck-at-0 Logic.....                                    | 5 |
| 2.2.3 Execution Cycle and Logging.....                         | 5 |
| 2.2.4 Environment and Tooling .....                            | 6 |
| 2.3 Instruction Corruption Fault Injection .....               | 6 |
| 2.3.1 Injection Logic .....                                    | 6 |
| 2.3.2 Execution Loop and Safety Control.....                   | 6 |
| 2.3.3 Environment and Tooling .....                            | 6 |
| 3 Reference Behavior Overview .....                            | 6 |
| 3.1 Bit-Flip Faults .....                                      | 6 |
| 3.1.1 Single-Event Upset Model .....                           | 6 |
| 3.1.2 Expected Register Outcomes .....                         | 7 |
| 3.2 Stuck-At Faults .....                                      | 7 |
| 3.2.1 Fault Model Definition.....                              | 7 |
| 3.2.2 Expected System Response.....                            | 7 |
| 3.3 Instruction Corruption Faults .....                        | 7 |
| 3.3.1 Corruption Impact on Instruction Stream .....            | 7 |
| 3.3.2 Expected Exception and Crash Modes.....                  | 7 |
| 4 Analysis of Fault Manifestation Across Injection Models..... | 8 |
| 4.1 Bit-Flip Fault Manifestation.....                          | 8 |
| 4.1.1 Injection Methodology .....                              | 8 |
| 4.1.2 Runtime Behavior .....                                   | 8 |
| 4.1.3 Fault Impact Scope.....                                  | 8 |
| 4.1.4 Outcome Summary .....                                    | 8 |
| 4.2 Stuck-at-Fault Manifestation.....                          | 8 |
| 4.2.1 Injection Methodology .....                              | 8 |
| 4.2.2 Runtime Behavior .....                                   | 8 |
| 4.2.3 Fault Propagation .....                                  | 9 |
| 4.2.4 Outcome Summary .....                                    | 9 |
| 4.3 Instruction Corruption Fault Manifestation .....           | 9 |

|                                                                        |    |
|------------------------------------------------------------------------|----|
| 4.3.1 Injection Methodology .....                                      | 9  |
| 4.3.2 Runtime Behavior .....                                           | 9  |
| 4.3.3 Fault Impact Characteristics.....                                | 9  |
| 4.3.4 Fault Severity Assessment .....                                  | 9  |
| 4.3.5 Outcome Summary .....                                            | 9  |
| 4.4 Fault Model Comparison .....                                       | 10 |
| 5 Comparative Fault Behavior Analysis .....                            | 10 |
| 5.1 Bit-Flip Faults .....                                              | 10 |
| 5.1.1 Comparison with SEU-Based Reference Model.....                   | 10 |
| 5.1.2 Conformance and Anomalies.....                                   | 10 |
| 5.1.3 Conclusion .....                                                 | 10 |
| 5.2 Stuck-at Faults .....                                              | 10 |
| 5.2.1 Comparison with Persistent Fault Reference Model.....            | 10 |
| 5.2.2 Detected Deviation from Recovery Mechanisms.....                 | 11 |
| 5.2.3 Conclusion .....                                                 | 11 |
| 5.3 Instruction Corruption Faults .....                                | 11 |
| 5.3.1 Comparison with Instruction Stream Reference Behavior.....       | 11 |
| 5.3.2 Deviations from Expected Fault-Handling Paths.....               | 11 |
| 5.3.3 Conclusion .....                                                 | 11 |
| 5.4 Summary of Observed vs. Reference Behavior .....                   | 11 |
| 6 System Resilience Assessment .....                                   | 12 |
| 6.1 Bit-Flip Faults: Transient and Maskable.....                       | 12 |
| 6.1.2 Resilience Evaluation.....                                       | 12 |
| 6.2 Stuck-At Faults: Persistent and Disruptive .....                   | 12 |
| 6.2.1 Failure Localization and Late Detection .....                    | 12 |
| 6.2.2 Resilience Evaluation.....                                       | 12 |
| 6.3 Instruction Corruption Faults: Catastrophic and Unrecoverable..... | 12 |
| 6.3.1 Instruction Fetch and Emulator Termination .....                 | 12 |
| 6.3.2 Resilience Evaluation.....                                       | 12 |
| 6.4 Fault Impact and Recovery Strength.....                            | 13 |
| 6.5 Overall System Robustness Evaluation .....                         | 13 |
| 6.5.1 Final Observation.....                                           | 13 |
| 7 Discussion .....                                                     | 13 |
| 7.1 Implications for RISC-V Resilience .....                           | 13 |
| 7.2 Limitations of the Emulation-Based Approach.....                   | 13 |
| 7.3 Recommendations for Hardware and Software Hardening.....           | 13 |
| 7.4 Faults in Automotive LiDAR Systems and Mitigation.....             | 13 |
| 7.5 Simulation vs. Real-Hardware Fault Variation.....                  | 13 |
| 8 Conclusion .....                                                     | 14 |
| 8.1 Summary of Findings.....                                           | 14 |

|                                                        |    |
|--------------------------------------------------------|----|
| 8.2 Contributions to Fault-Injection Methodology ..... | 14 |
| 8.3 Future Work .....                                  | 14 |
| Annex .....                                            | 14 |
| References .....                                       | 15 |

## **Abstract**

This report evaluated RISC-V system resilience under three fault-injection models within a QEMU emulator. It implemented automated bit-flip, stuck-at, and instruction-corruption campaigns via GDB and Python to induce controlled hardware faults. The study defined expected behaviors from established SEU, stuck-at, and instruction-corruption reference models and then measured actual system responses during early boot and init routines. It observed complete execution continuity under transient bit-flip faults, emergency-mode hangs for persistent stuck-at errors, and immediate emulator termination for instruction-stream corruption. The comparative analysis revealed a high tolerance for transient register faults, moderate robustness to persistent register-errors, and minimal recoverability from instruction-level corruption. The findings demonstrate critical resilience gaps in exception-path coverage and suggest targeted hardening strategies, ECC integration, control-flow integrity checks, and redundant instruction encoding—to improve RISC-V fault tolerance in safety-critical embedded applications.

## **1 Introduction**

Fault tolerance in modern embedded systems, especially those operating in safety-critical or remote environments, is not merely a design goal but has become integral. From aerospace control systems to autonomous vehicles and industrial automation, embedded devices must operate reliably despite exposure to environmental disturbances, hardware degradation, and software anomalies (Koopman, 2016). As the hardware complexity and software stack deepen, fault detection, containment, and recovery require rigorous testing and validation strategies. Among these, fault injection has emerged as a cost-effective and reproducible method to study fault propagation and system-level resilience (Koopman, 2016).

Fault injection enables engineers to simulate realistic hardware and software faults without physical tampering. In virtualized environments like QEMU, developers can introduce controlled faults into memory, instruction flow, or register state and then observe how the system responds (Bellard, 2005). This emulation makes fault injection especially relevant in emulated RISC-V systems, where early-stage testing is critical due to the architecture's adoption in research and low-cost commercial systems.

However, fault injection is only meaningful when paired with robust system behavior analysis and validation. System behavior and analysis involve defining expected behavior based on reference models or known-correct output, executing fault-injection campaigns, and comparing the system's response. Such validation uncovers whether the system handles errors gracefully (e.g., failsafe state, graceful degradation) or catastrophically (e.g., kernel panic, undefined state). For this reason, capturing system metrics, like register values, memory integrity, exception logs, or execution halts, becomes essential. Without this feedback loop, injected faults cannot inform resilience strategies (Ziegler & Lanford, 1979; Abramovici et al., 1994).

This report extends prior work involving bit-flip, stuck-at, and instruction corruption faults injected into a QEMU-simulated RISC-V system. It aims to document and validate the system's behavior under these injected faults against reference outcomes. The objective is to identify whether the observed deviations align with the expected fault model behavior, whether fault masking occurs, or whether system failure pathways emerge (Koppenstein, 2020). This report tries to contribute to forming a repeatable, architecture-agnostic approach to validating virtualized fault tolerance mechanisms using open-source tools, particularly in Edge AI contexts where on-demand deep neural network inference acceleration via edge computing introduces additional robustness challenges (Li et al., 2020).

## **2 Fault Injection Methodology**

This section documents the previously implemented methodology for injecting hardware-level faults into a QEMU-emulated RISC-V system. The report categorized the faults into bit-flip, stuck-at, and instruction corruption. The scripts injected faults using *gdb-multiarch* through a GDB Machine Interface (MI3) and automated control via Python. Each fault campaign relied on consistent CPU interruption to ensure controlled injection and accurate data collection.

### **2.1 Bit-Flip Fault Injection**

The bit-flip fault model simulated random transient upsets in register values, commonly caused by environmental noise or radiation effects.

### 2.1.1 Injection Logic

The fault injection script read the current value of each target register using the GDB command `-data-evaluate-expression $<reg>`. It then applied a bitwise XOR operation to flip a specific bit:

$$flipped\_val = current\_val \wedge (1 \ll bit\_to\_flip)$$

Finally, it committed the corrupted value back to the register via:

$$set \$<reg> = <flipped\_val>$$

### 2.1.2 Automation and Execution

The script iterated through a defined register set (x10, x11, x12, etc.) across multiple cycles. It issued an interrupt command before each injection to pause the CPU safely and maintain control consistency. A time delay between injections allowed the target system to react, while the report logged the original and the modified register states for every iteration.

### 2.1.3 Environment and Tooling

- **Fault target:** pc, x1, x2, x8, x9
- **Injection count:** 5 iterations per run
- **Tools used:**
  - QEMU (RISC-V target)
  - gdb-multiarch in MI3 mode
  - Python 3 with pygdbmi
- **Fault control point:** At runtime, via the interrupt and resume loop

## 2.2 Stuck-at Fault Injection

This fault model introduced permanent logical faults by forcing a specific bit in the register to remain either high (1) or low (0), simulating hardware-level line failures.

### 2.2.1 Stuck-at-1 Logic

The script forced bit 63 high using a bitwise OR operation:

$$stuck\_val = current\_val \vee (1 \ll 63)$$

It applied the result back to the register using:

$$set \$<reg> = <stuck\_val>$$

### 2.2.2 Stuck-at-0 Logic

To simulate a low-level stuck-at fault, the script cleared bit 63 using a bitwise AND with the inverse mask:

$$stuck\_val = current\_val \wedge \sim(1 \ll 63)$$

It then issued a set command to update the register.

### 2.2.3 Execution Cycle and Logging

The injector ran 25 iterations for each mode (stuck-at-1, stuck-at-0) for each register. The report logged the initial value and the modified value after injection. An interrupt consistently paused the CPU, with up to three retry attempts for robust fault delivery. Each step included a delay for monitoring the system's reaction.

### 2.2.4 Environment and Tooling

- **Fault targets:** x1, x2, x9, x8, pc
- **Total injections:** 25 stuck-at-1 + 25 stuck-at-0 per register
- **Control:** CPU interrupt + fault set + delay
- **Logging:** All iterations saved to Word documents for post-analysis

## 2.3 Instruction Corruption Fault Injection

Instruction corruption faults targeted executable memory, flipping a bit in an instruction word to simulate memory corruption at the software code level.

### 2.3.1 Injection Logic

The script read the 4-byte instruction at a fixed address (0x80000000) using the GDB MI3 memory-read command:

```
-data-read-memory 0x80000000 x 1 4 4 1
```

After extracting the instruction word, the script applied a bit flip at the defined bit position:

$$corrupted = original \wedge (1 \ll bit\_to\_flip)$$

It wrote the corrupted instruction back to memory using:

```
set {int}0x80000000 = <corrupted>
```

### 2.3.2 Execution Loop and Safety Control

The script performed 50 fault injections. Before each injection, it issued an interrupt to halt execution and ensure consistent read-write memory access. Each iteration logged the original instruction and its corrupted variant at each step. A delay followed each injection to allow fault effects to manifest.

### 2.3.3 Environment and Tooling

- **Target address:** 0x80000000 (valid instruction site)
- **Bit flipped:** Bit 63
- **Iteration count:** 10
- **Tools used:**
  - QEMU RISC-V
  - gdb-multiarch
  - pygdbmi in Python
- **Safety mechanisms:** Interrupt with retry; error handling for read failures

## 3 Reference Behavior Overview

This section defines the expected system response for each fault model injected in Section 2. It bases these definitions on established fault-injection literature and hardware-testing standards to determine the correct behavior for validating observed outputs.

### 3.1 Bit-Flip Faults

#### 3.1.1 Single-Event Upset Model

The report adopts the Single-Event Upset (SEU) fault model, in which a single stored bit flips its value due to an ionizing particle strike or electromagnetic disturbance (Huang & Jiang, 2019). SEUs induce only transient errors: the system restores the usual operation once the upset propagates through the architectural state without leaving permanent damage (Elder et al., 1988)

### 3.1.2 Expected Register Outcomes

The reference behavior dictates that each register read, modified by a single-bit toggle, differs from its original value by exactly one bit. The system should:

- Continue execution if the flipped bit does not corrupt control flow or critical state fields.
- Raise a trap or fault exception if the flipped bit causes an illegal instruction or privilege violation.
- Produce silent data corruption (SDC) if the flipped bit alters application data without triggering an exception (Huang & Jiang, 2019).

The report treats any deviation from these three categories as an anomaly in the fault-handling mechanism

## 3.2 Stuck-At Faults

### 3.2.1 Fault Model Definition

The single stuck-at-fault model assumes a line or bit in a register remains fixed at logic '1' (stuck-at-1) or logic '0' (stuck-at-0) throughout execution (Cottrell, 2018). This model represents manufacturing defects in transistors or interconnect layers that cannot switch state under any stimulus.

### 3.2.2 Expected System Response

Reference behavior requires the system to:

- Detect the discrepancy when the stuck bit appears in circuit outputs or architectural state, typically via in-hardware ECC or software consistency checks.
- Signal an error (machine check, exception) if the stuck bit resides in a control-state field (e.g., program counter, privilege bits).
- Mask or correct the error if the stuck bit falls within ECC-protected memory or register regions without affecting visible program output (Huang & Jiang, 2019).

The report flags any silent masking in unprotected fields or failures to trap control-field faults as deviations from expected behavior.

## 3.3 Instruction Corruption Faults

### 3.3.1 Corruption Impact on Instruction Stream

Instruction corruption faults flip a bit within the 32-bit opcode or operand fields. Literature indicates three broad outcomes:

- **Expected:** The corrupted instruction decodes to a different valid instruction, and the system executes it by possibly altering program semantics.
- **Trap:** The corrupted bit produces an illegal opcode or misaligned access, triggering an exception.
- **Crash:** The corrupted instruction leads to undefined behavior, system hang, or reboot (Rivière et al., 2015).

### 3.3.2 Expected Exception and Crash Modes

The reference behavior classifies each instruction corruption as follows:

- **Valid-to-Valid Transformation:** The instruction decoder maps the corrupted word to a different but legal instruction; execution continues along a new control path.
- **Invalid-to-Exception:** The decoder identifies an illegal or unsupported opcode and raises an illegal-instruction trap.
- **Invalid-to-Fault:** Corruption in privilege-level or alignment bits causes a fault handler invocation or machine-check event (Roschian et al., 2015).

The report expects the fault-injection harness to observe one of these outcomes per injection; any multi-fault behavior (e.g., silent skip of several instructions) indicates a discrepancy in the infusion or emulation platform.

## 4 Analysis of Fault Manifestation Across Injection Models

This section analyzes the observable impact of fault injections on a QEMU-emulated RISC-V platform executing a modern Linux distribution. The evaluation focused on fault types with increasing persistence and severity: bit-flip, stuck-at, and instruction corruption. Each test targeted control-relevant registers or kernel text memory and recorded log outputs and emulation behavior under constrained, reproducible scenarios.

### 4.1 Bit-Flip Fault Manifestation

#### 4.1.1 Injection Methodology

The injection script executed bitwise XOR operations on bit 63 of selected general-purpose registers (GPRs). The script attached itself to QEMU's GDB stub during the early boot stage and toggled the bit value dynamically. The injection occurred before the control transfer to the init system (*systemd*), with registers chosen for their relevance to stack handling, return paths, and *syscall* interfaces (x1, x2, x8, x9).

#### 4.1.2 Runtime Behavior

The Linux kernel completed all boot stages successfully following bit-flip injection. It mounted the root filesystem, initialized the scheduler, and launched *systemd* targets without emitting kernel panics or warnings. Neither the journal logs nor the serial console reported misbehavior. The system transitioned into multi-user mode, maintaining complete operational integrity.

#### 4.1.3 Fault Impact Scope

The flipped bit remained ineffective during injection or resided in an overwritten register before critical path execution. The MSB, being the sign bit in a 64-bit architecture, flipped the value's sign but did not result in address faults or invalid instructions. The kernel likely masked or ignored the upper 32 bits during early logic stages, reducing the propagation scope of the injected corruption.

#### 4.1.4 Outcome Summary

The system tolerated the injected bit-flip with no observable malfunction. Transient errors introduced during early execution stages remained non-persistent and non-propagating.

### 4.2 Stuck-at-Fault Manifestation

#### 4.2.1 Injection Methodology

The stuck-at-fault script forced bit 63 of critical registers to remain at logic high (1) or low (0). Unlike bit-flip faults, the corruption persisted across subsequent instructions, simulating manufacturing-time or radiation-induced defects. Registers with a higher control-flow influence, such as x2 (stack pointer) and *syscall*-related GPRs, received forced values during system initialization.

#### 4.2.2 Runtime Behavior

The kernel initiated standard device setup, reached filesystem mounting routines, and failed to access */boot/efi*. *Systemd* emitted the following log:

```
[FAILED] Failed to mount boot-efi.mount - /boot/efi.  
Dependency failed for local-fs.target - Local File Systems.
```

Following this failure, the system entered emergency mode (*emergency.target*). *Systemd* halted all dependent targets, and no user-space shell or service executed correctly. The emergency shell appeared as a degraded fallback.

### 4.2.3 Fault Propagation

Persistent corruption in x2 or x9 introduced malformed stack frames or corrupted *syscall* arguments. The filesystem mount routine received invalid pointers or paths, causing internal kernel rejections. Since bit 63 functions as a sign bit, corrupted values misrepresented memory offsets or device identifiers as negatives or inaccessible. These conditions triggered non-recoverable logic failures in critical early-stage subsystems.

### 4.2.4 Outcome Summary

Stuck-at-faults induced persistent, propagating corruption. Unlike bit-flips, these faults remained active throughout execution, compromising memory alignment, stack integrity, and system call correctness. The system could not progress past the init sequence.

## 4.3 Instruction Corruption Fault Manifestation

### 4.3.1 Injection Methodology

The injection framework iteratively accessed executable memory in 4-byte steps and applied bitwise corruption via XOR. The script modified instruction encodings in-place, including opcodes, immediate values, and register specifiers. The FLIP\_MASK parameter selectively flipped bits to maximize control-flow and semantic corruption.

### 4.3.2 Runtime Behavior

Following the corruption of multiple instructions in the kernel code section, QEMU forcibly halted execution. The terminal output displayed:

*qemu-system-riscv64: QEMU: Terminated via GDBstub*

No kernel panic occurred, and no journal logs indicated the cause. The system did not enter emergency mode or emit error traces. The emulation halted before complete initialization and the serial output truncated mid-boot.

### 4.3.3 Fault Impact Characteristics

Corrupted instructions included undefined opcodes, illegal CSR accesses, and malformed branches. Since instruction memory holds privileged executable code, even a malformed word caused architectural violations. The QEMU emulator could not recover from these exceptions and terminated the process without passing control back to the guest kernel.

### 4.3.4 Fault Severity Assessment

Unlike register faults, instruction corruption modified persistent logic. Corrupted instructions bypassed recovery by fault handlers or exception traps. The RISC-V instruction set lacks runtime opcode redundancy, so even minor corruption resulted in decoding failures or privilege errors.

### 4.3.5 Outcome Summary

Instruction faults produced the most severe disruption. They caused immediate, unrecoverable emulation collapse. The location of the fault in static memory magnified its impact by altering the kernel's precompiled logic.

## 4.4 Fault Model Comparison

| Aspect                   | Bit-Flip (MSB) | Stuck-at (MSB)  | Instruction Corruption |
|--------------------------|----------------|-----------------|------------------------|
| Persistence              | Transient      | Permanent       | Permanent              |
| Fault Medium             | GPRs           | GPRs            | GPRs                   |
| Observable Behavior      | None           | Emergency Shell | Emulator Termination   |
| Kernel Panic             | No             | No              | No                     |
| Recoverability           | Full           | Partial         | None                   |
| Fault Timing Sensitivity | High           | Low             | Irrelevant             |
| Disruption Severity      | Low            | High            | Critical               |

Table 1-Comparison table of the fault models

## 5 Comparative Fault Behavior Analysis

This section contrasts the observed outcomes from experimental fault injections with their corresponding reference models. Each subsection evaluates whether system behavior aligns with theoretical expectations defined in Section 3. This report analyzed deviations against system architecture, instruction decoding, and fault propagation dynamics.

### 5.1 Bit-Flip Faults

#### 5.1.1 Comparison with SEU-Based Reference Model

The observed bit-flip behavior aligns with the Single-Event Upset (SEU) model's definition of transient register faults. All injections targeted the most significant bit (bit 63) of general-purpose registers and exhibited no permanent impact on system control flow.

Reference behavior predicts three valid outcomes for SEUs: continued execution, exception trapping, or silent data corruption (SDC). The experiment consistently showed the first and complete execution continuity without kernel panic or visible error propagation.

#### 5.1.2 Conformance and Anomalies

The system met reference criteria by successfully masking or overwriting corrupted register values before their usage in critical routines. Bit-63 flipping did not corrupt addresses or flags in a way that triggered exception handling or caused observable SDC.

Although no trap or panic occurred, this absence does not constitute a deviation. Instead, it suggests that the injected faults failed to intersect the live architectural state or sensitive control logic during the boot window.

#### 5.1.3 Conclusion

Experimental results conform to reference behavior for SEU-induced bit-flips, confirming their transient, low-risk nature under selected injection timing.

### 5.2 Stuck-at Faults

#### 5.2.1 Comparison with Persistent Fault Reference Model

Stuck-at faults exhibited persistent behavior in line with the reference model's assumptions. These faults permanently forced bit 63 in selected registers to logic high or low, causing sustained register corruption across instruction cycles. According to reference behavior, the system should trap faults in control-critical fields and continue if the fault resides in masked or corrected locations. In the experiment, the system failed to mount */boot/efi*, entered *emergency.target*, and terminated further *init* routines.

### 5.2.2 Detected Deviation from Recovery Mechanisms

The system failed to raise an explicit trap or illegal-instruction exception despite remaining in a sustained invalid control state. Instead, execution proceeded until a corrupted system call or address pointer caused a resource mount failure, cascading into a boot halt.

This behavior partially deviates from expected reference handling. Rather than triggering a machine check or early exception, the system deferred fault manifestation until deeper into *init-stage* routines.

The likely cause is that ECC or software-level checks did not cover GPR corruption, especially in *syscall* argument registers (x10–x17). As a result, fault detection occurred indirectly via secondary failures.

### 5.2.3 Conclusion

Stuck-at faults behaved disruptively as expected, but recovery mechanisms underperformed in early-stage detection and containment, representing a partial deviation from reference behavior.

## 5.3 Instruction Corruption Faults

### 5.3.1 Comparison with Instruction Stream Reference Behavior

Instruction corruption faults exhibited the most severe deviations. While the reference model anticipates a valid instruction remapping or trap generation, the experiment triggered QEMU termination without observable kernel output or panic trace.

This behavior aligns with the "crash" outcome described in the literature but exceeds typical software-level failure by halting the emulator. The GDBstub termination log indicated that the emulator stopped execution immediately after fetching a corrupted instruction.

### 5.3.2 Deviations from Expected Fault-Handling Paths

The system did not invoke an illegal-instruction handler or fault recovery routine within the kernel. Instead, corrupted instruction fetches caused QEMU to abort execution without exception redirection.

This behavior suggests that the injected faults corrupted instruction fields beyond ISA-decodable boundaries—possibly breaking instruction alignment, opcode format, or privilege bits. The fault exceeded RISC-V exception coverage, triggering an unrecoverable state within QEMU's simulation core.

From a reference standpoint, the observed behavior falls into the "invalid-to-fault" class. However, the absence of OS-level diagnostics or panic logs deviates from expectations that assume the kernel will handle such faults.

### 5.3.3 Conclusion

Instruction corruption behaved more destructively than modeled. While conforming to the "crash" category, it deviated from expected kernel exception-handling pathways by terminating QEMU directly.

## 5.4 Summary of Observed vs. Reference Behavior

| Fault Type                 | Reference Model Outcome                       | Observed Behavior                         | Conformance |
|----------------------------|-----------------------------------------------|-------------------------------------------|-------------|
| Bit-Flip (MSB)             | Transient, recoverable; may cause trap or SDC | Full execution with no observable effects | Full        |
| Stuck-at (MSB)             | Persistent; trap on control-state corruption  | Emergency mode after delayed failure      | Partial     |
| Instruction Set Corruption | Trap, crash or altered instruction semantics  | Immediate QEMU abort, no panic or handler | Partial     |

Table 2- Summary of Observed and Reference Behavior

## **6 System Resilience Assessment**

This section synthesizes recovery, failure, and exception behavior across all fault models to evaluate system resilience. It combines experimental observations with theoretical expectations to assess fault impact, recovery coverage, and overall architectural robustness.

### **6.1 Bit-Flip Faults: Transient and Maskable**

#### **6.1.1 Execution Continuity and Error Masking**

Bit-flip injections targeted the most significant register bit during the early boot stages. The system completed initialization without panics or error logs (Huang & Jiang, 2019). The kernel overwrote or ignored corrupted MSBs before they influenced critical execution.

#### **6.1.2 Resilience Evaluation**

This behavior confirmed temporal masking and register overwrite protection (Elder et al., 1988). The platform tolerated transient register faults injected outside sensitive control-flow windows. Bit-flip faults generated no observable fault traces.

### **6.2 Stuck-At Faults: Persistent and Disruptive**

#### **6.2.1 Failure Localization and Late Detection**

Stuck-at faults forced bit 63 to logic high or low across all subsequent instructions. The kernel attempted to mount /boot/efi, then entered emergency mode after mount failure (Cottrell, 2018). The system halted further init sequences.

#### **6.2.2 Resilience Evaluation**

This outcome highlighted insufficient early detection of persistent register faults (Huang & Jiang, 2019). ECC and software checks did not cover general-purpose registers. The system only detected errors through secondary filesystem failures.

### **6.3 Instruction Corruption Faults: Catastrophic and Unrecoverable**

#### **6.3.1 Instruction Fetch and Emulator Termination**

The injector corrupted opcode bits in kernel text memory via in-place XOR. QEMU terminated execution immediately after fetching a malformed instruction (Rivière et al., 2015). The emulator produced no kernel panic or trace before aborting.

#### **6.3.2 Resilience Evaluation**

Instruction corruption breached all OS-level exception paths (Roschian et al., 2015). The system invoked no illegal-instruction trap. Emulation stopped at the simulator layer, indicating zero recoverability.

## 6.4 Fault Impact and Recovery Strength

| Fault Model            | Persistence | Detection Stage | System Response     | Resilience Level |
|------------------------|-------------|-----------------|---------------------|------------------|
| Bit-Flip (MSB)         | Transient   | Not Detected    | Execution Continues | High             |
| Stuck-at (MSB)         | Permanent   | Post-failure    | Emergency Mode      | Medium           |
| Instruction Corruption | Permanent   | Emulator-Level  | Immediate abort     | Low              |

Table 3-Fault Impact and Recovery Strength

## 6.5 Overall System Robustness Evaluation

The experiments demonstrated asymmetric resilience across fault domains. The system masked transient bit-flip errors effectively (Elder et al., 1988). It exhibited delayed detection for stuck-at faults, relying on secondary failure signals (Cottrell, 2018). Instruction corruption faults bypassed all kernel traps, collapsing the emulator (Rivière et al., 2015; Roscian et al., 2015).

**6.5.1 Final Observation** The platform maintains a high tolerance for transient register faults but shows critical vulnerabilities to persistent and instruction-level faults.

## 7 Discussion

This section interprets resilience findings, acknowledges emulation limitations, and proposes hardening strategies.

### 7.1 Implications for RISC-V Resilience

The resilience analysis highlighted strong masking of transient faults under early execution windows. RISC-V architectures demonstrated high tolerance for isolated bit-flip errors (Huang & Jiang, 2019). However, persistent faults breached control-flow integrity and triggered systemic failures (Cottrell, 2018). Instruction-level corruption produced unrecoverable states, revealing critical gaps in exception-path coverage (Rivière et al., 2015).

### 7.2 Limitations of the Emulation-Based Approach

Emulation platforms cannot perfectly model real-life hardware timing or analog effects. QEMU's virtual GDB stub lacks precise timing granularity and analog fault modeling (Bellard, 2005). Fault injection via software scripts cannot reproduce voltage-induced metastability or electromagnetic interference inherent to physical chips. Consequently, observed fault behavior may diverge from hardware deployments under real environmental stressors.

### 7.3 Recommendations for Hardware and Software Hardening

Designers should integrate ECC on general-purpose registers and caches to detect and correct bit-flips before they spread (Elder et al., 1988). Control-flow integrity checks and redundant instruction encoding can mitigate instruction corruption faults (Roscian et al., 2015). Software frameworks must include early-stage "sanity checks" on syscall arguments and pointer validity to contain stuck-at faults before critical resource access.

### 7.4 Faults in Automotive LiDAR Systems and Mitigation

Automotive LiDAR units commonly face stuck-at and transient bit-flip faults due to cosmic radiation and temperature cycling (Ziegler & Lanford, 1979). Stuck-at faults in data converters can distort range measurements, while transient upsets may introduce spurious reflections. Designers mitigate these issues by employing triple-modular redundancy in ADC channels and adaptive majority-voting logic on distance measurements (Huang & Jiang, 2019). Periodic register scrubbing and runtime health monitoring also reduce latent fault accumulation.

### 7.5 Simulation vs. Real-Hardware Fault Variation

Hardware implementations may exhibit fault behaviors beyond those seen in QEMU. Process-node variations, analog circuit aging, and power-rail noise can induce multi-bit upsets and timing violations irreproducible in emulation (Dodd & Massengill, 2003). Simulation still provides valuable functional insights, but validation on silicon or FPGA prototypes remains essential to capture analog fault effects and verify complete system robustness.

## 8 Conclusion

This section summarizes key findings, highlights methodological contributions, and outlines directions for further research.

### 8.1 Summary of Findings

The experiments confirm that early-stage register overwrites actively suppress transient bit-flip faults, preserving the system's boot integrity (Huang & Jiang, 2019). Persistent stuck-at faults caused emergency-mode failures due to undetected control-state corruption (Cottrell, 2018). Instruction corruption faults generated unrecoverable emulator-level crashes by breaching static code integrity (Rivière et al., 2015).

### 8.2 Contributions to Fault-Injection Methodology

The report established a repeatable, script-driven fault-injection framework using QEMU and pygdbmi that automated bit-flip, stuck-at, and instruction corruption campaigns. It introduced a structured comparison against reference fault models to quantify resilience gaps. The analysis highlighted the need for standardized injection timing, detailed logging, and systematic conformance checks to validate fault-tolerance strategies (Koopman, 2016).

### 8.3 Future Work

Subsequent steps include integrating ECC protection for general-purpose registers to detect and correct bit-flips in situ (Dodd & Massengill, 2003). The methodology will extend to memory-bus and cache-line fault models, capturing multi-bit upsets and timing violations. The framework will undergo hardware-in-the-loop validation on FPGA-based RISC-V prototypes to evaluate analog and process-variation effects beyond emulation limitations.

## Annex

Please refer to the attached .pdf file containing console logs, python scripts, and result from injecting faults.

## References

- [1] P. Koopman, “Understanding fault injection,” *IEEE Security & Privacy*, vol. 14, no. 4, pp. 71–75, Jul.–Aug. 2016.
- [2] F. Bellard, “QEMU, a fast and portable dynamic translator,” in *Proc. USENIX Annu. Tech. Conf.*, 2005.
- [3] J. F. Ziegler and W. A. Lanford, “Effect of cosmic rays on computer memories,” *Science*, vol. 206, no. 4420, pp. 776–788, Nov. 1979.
- [4] M. Abramovici, M. A. Breuer, and A. D. Friedman, *Digital Systems Testing and Testable Design*. Rockville, MD, USA: Computer Science Press, 1994.
- [5] M. Koppenstein, “Fast user-mode fault injection with FAIL\*,” M.S. thesis, Dept. Comput. Sci., TU Dortmund Univ., Dortmund, Germany, 2020.
- [6] D. Li, Z. Zhou, and X. Chen, “Edge AI: On-demand accelerating deep neural network inference via edge computing,” *IEEE Trans. Wireless Commun.*, vol. 19, no. 1, pp. 447–457, Jan. 2020.
- [7] A. Elder, T. Loveless, R. Schrimpf, and D. Fleetwood, “Single event upset in microelectronics,” *IEEE Trans. Nucl. Sci.*, vol. 35, no. 6, pp. 1659–1664, Dec. 1988, doi: 10.1109/23.13287.
- [8] Q. Huang and J. Jiang, “An overview of radiation effects on electronic devices under severe accident conditions in NPPs, rad-hardened design techniques and simulation tools,” *Prog. Nucl. Energy*, vol. 114, pp. 105–120, 2019.
- [9] D. R. Cottrell, “Design automation technology roadmap,” in *Computer Aided Design and Design Automation*, W.-K. Chen, Ed. Boca Raton, FL, USA: CRC Press, 2018, pp. 1–10.
- [10] D. Rivière, B. De Sutter, and B. Robisson, “On the sensitivity of ARM and x86 processors to instruction skip faults,” in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Grenoble, France, 2015, pp. 635–638, doi: 10.7873/DATE.2015.1127.
- [11] C. Roscian, A. Moradi, and A. Poschmann, “Leakage detection with high-dimensional statistical analysis,” in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Grenoble, France, 2015, pp. 107–112, doi: 10.7873/DATE.2015.0442.
- [12] P. E. Dodd and L. W. Massengill, “Basic mechanisms and modeling of single-event upset in digital microelectronics,” *IEEE Trans. Nucl. Sci.*, vol. 50, no. 3, pp. 583–602, Jun. 2003, doi: 10.1109/TNS.2003.813129.